Task 1. Create a Node.js Express application

Create a new GitHub repository:

`playlist-app`

Project structure:

```
None
playlist-app/
│
├── app.js
├── package.json
├── Dockerfile
└── .github/
    └── workflows/
        └── deploy.yml
```

app.js

```
None
const express = require("express");

const app = express();

app.get("/", (req, res) => {
    res.send("Welcome to My Playlist");
});

app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

package.json

```
None
{
```

```
  "name": "playlist-app",
  "version": "1.0.0",
  "description": "My Playlist Application",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Dockerfile

```
None
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install --omit=dev

COPY app.js .

EXPOSE 3000

CMD ["npm", "start"]
```

Build Docker image:

```
None
docker build -t playlist-app .
```

Run container:

```
None
docker run -d -p 3000:3000 --name playlist-container playlist-app
```

Open:

Open Playlist Application

Expected output:

```
None
Welcome to My Playlist
```

Task 2. Push Docker image to Docker Hub

Login:

```
None
docker login
```

Tag the image:

```
None
docker tag playlist-app YOUR_DOCKERHUB_USERNAME/playlist-app:latest
```

Push:

```
None
docker push YOUR_DOCKERHUB_USERNAME/playlist-app:latest
```

Verify your image on Docker Hub.

Task 3. Create GitHub Actions workflow

Create:

`.github/workflows/deploy.yml`

None

```
name: Playlist App CI/CD

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and Push Image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME
}}/playlist-app:latest
```

Push workflow:

None

```
git add .
git commit -m "Add playlist CI CD workflow"
git push origin main
```

Task 4. Deploy the application to Kubernetes

First, create:

`playlist-deployment.yaml`

```
None
apiVersion: apps/v1
kind: Deployment

metadata:
  name: playlist-deployment

spec:
  replicas: 2

  selector:
    matchLabels:
      app: playlist

  template:
    metadata:
      labels:
        app: playlist

    spec:
      containers:
        - name: playlist-container
          image: YOUR_DOCKERHUB_USERNAME/playlist-app:latest
          ports:
            - containerPort: 3000
```

Create a Service:

`playlist-service.yaml`

```
None
apiVersion: v1
```

```
kind: Service

metadata:
  name: playlist-service

spec:
  type: NodePort

  selector:
    app: playlist

  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30081
```

Apply manually for the initial deployment:

```
None
kubectl apply -f playlist-deployment.yaml

kubectl apply -f playlist-service.yaml
```

Verify:

```
None
kubectl get deployments
kubectl get pods
kubectl get services
```

Now update the GitHub Actions workflow to deploy automatically.

Add a GitHub Secret:

`KUBE_CONFIG`

Its value should be the Base64-encoded kubeconfig for your deployment cluster.

For a suitable Kubernetes cluster with a securely configured GitHub Actions runner, use this updated workflow:

None

```
name: Playlist App CI/CD

on:
  push:
    branches:
      - main

jobs:
  build-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Docker Hub Login
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and Push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/playlist-app:${{
github.sha }}

      - name: Configure Kubernetes
        run: |
          mkdir -p $HOME/.kube
          echo "${{ secrets.KUBE_CONFIG }}" | base64 -d >
$HOME/.kube/config
```

```
      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/playlist-deployment \
            playlist-container=${{ secrets.DOCKERHUB_USERNAME
}}/playlist-app:${{ github.sha }}

          kubectl rollout status deployment/playlist-deployment
```

Important practical note: This final workflow requires a reachable Kubernetes cluster and valid GitHub Actions credentials. A local Minikube cluster running on your laptop is not automatically accessible to GitHub-hosted runners. For a real cloud deployment, use EKS, GKE, or AKS with appropriately restricted access. Also, configure GitHub OIDC where supported rather than relying on long-lived credentials.

Task 5. Two improvements for CI/CD reliability and security

Improvement 1: Use secure credentials

- Store Docker Hub credentials in GitHub Secrets.
- Never hardcode passwords or access tokens in YAML files.
- Use limited-permission access tokens.
- Prefer cloud OIDC authentication instead of permanent cloud credentials.

Implementation: Store sensitive values in GitHub Secrets and reference them using `${{ secrets.SECRET_NAME }}`.

Improvement 2: Use versioned Docker image tags

Instead of always using:

```
None
playlist-app:latest
```

Use unique tags:

```
None
playlist-app:<github-commit-sha>
```

Benefits:

- Easier rollback.
- Better version tracking.
- Prevents confusion between old and new images.
- Makes deployment more reliable.

Improvement 3: Add deployment verification

Add:

```
None
kubectl rollout status deployment/playlist-deployment
```